

RECONOCIMIENTO DE ELEMENTOS EN EL DESARROLLO DE UN PROGRAMA INFORMÁTICO

Reconocimiento de elementos en el desarrollo de un programa informático

Para crear un programa, intervienen varios elementos y herramientas que trabajan juntos.

1. Lenguaje de programación



Es el idioma que se usa para escribir las instrucciones del programa.

Ejemplos: Java, Python, C++, JavaScript, etc.



2. Entorno de desarrollo (IDE)

Es el programa donde se escribe, se edita y se prueba el código.

Ejemplos: Visual Studio Code, Eclipse, IntelliJ, NetBeans, etc.

3. Librerías y frameworks



Son conjuntos de código ya hechos que facilitan el desarrollo de ciertas funciones.

Ejemplos: React, Angular, Spring, Pandas, etc.

6. Compilador o intérprete



Traduce el código fuente a un lenguaje que la máquina pueda entender (o lo ejecuta directamente).

Ejemplos: javac, Python, Node.js, GCC, etc.

4. Herramientas de control de versiones



Permiten guardar los cambios del código y trabajar en equipo.

Ejemplos: Git, GitHub, GitLab, Bitbucket, etc.

7. Archivos y estructura del proyecto



Organizan el código, los recursos y la configuración del programa.

Ejemplos: .java, .py, .html, .css, carpetas src, resources, etc.

5. Sistema operativo



Es el sistema que gestiona los recursos del ordenador donde se ejecuta el programa.

Ejemplos: Windows, Linux, macOS, etc.

En resumen...



Código fuente
(lenguaje de programación)



Compilación / Interpretación
(traduce o ejecuta)



Ejecución
(en el sistema operativo)



Programa final
(funcionando)

INDICE

Contenido

Parte 1: Análisis teórico de conceptos.....	3
Explicación de los conceptos básicos.....	3
Clasificación de lenguajes de programación.....	7
Parte 2: Actividad práctica y de análisis	9
Identificación de paradigmas de programación a partir de ejemplos.....	9
Fragmento 1.....	10
Fragmento 2	11
Fragmento 3	12
Fragmento 4	12
Actividad en grupo	13

Objetivo: Evaluar la capacidad de los alumnos para reconocer los elementos y herramientas que intervienen en el desarrollo de un programa informático, diferenciando los conceptos de código fuente, objeto y ejecutable, así como clasificar los lenguajes de programación (imperativos y declarativos).

Parte 1: Análisis teórico de conceptos

1. Explicación de los conceptos básicos:

1.1 Describe en tus propias palabras qué es el código fuente, código objeto y código ejecutable.

Código fuente

El **código fuente** es el conjunto de instrucciones que escribe un programador utilizando un lenguaje de programación.

Es el código que podemos leer y modificar antes de que sea traducido para que el ordenador pueda ejecutarlo.

Por ejemplo, en Java:

```
public class HolaMundo {  
    public static void main(String[] args) {  
        System.out.println("Hola mundo");  
    }  
}
```

Este código es código fuente porque ha sido escrito por el programador.

El archivo podría llamarse:

HolaMundo.java

1.2 Código objeto

El **código objeto** es el resultado de traducir el código fuente mediante un compilador. Está más cerca del lenguaje que puede entender el ordenador y normalmente no está pensado para que una persona lo lea directamente.

Por ejemplo, en un lenguaje como C:

Código fuente

Compilador



Código objeto

El código objeto puede almacenarse en archivos como:

programa.oo, dependiendo del sistema y herramientas utilizadas: programa.obj

1.3 El código ejecutable

es el resultado que permite ejecutar un programa en un determinado sistema.

En Windows, por ejemplo, podemos encontrar ejecutables con extensión:

.exe

Un ejemplo sería:

calculadora.exe

Al abrirlo, el sistema operativo carga el programa y el procesador ejecuta sus instrucciones.

En Java hay una particularidad: normalmente el código fuente **.java** se compila a **bytecode .class**, que posteriormente es ejecutado por la **Máquina Virtual de Java (JVM)**.

Explica las fases por las que pasa un programa desde que es escrito en un lenguaje de programación hasta que es ejecutado en el procesador.

Código fuente (.java)



Compilador



Bytecode (.class)



JVM



Procesador

1.4 Fases programa

Cuando escribimos un programa, no pasa directamente del código que hemos escrito al procesador. Hay diferentes fases:

- 1. Escritura del código fuente

El programador escribe el programa utilizando un lenguaje de programación.

Por ejemplo:

```
int edad = 20;  
System.out.println(edad);
```

- 2. Análisis léxico

El programa analiza los elementos individuales del código, llamados **tokens**.

Por ejemplo:

```
int edad = 20;
```

Se pueden identificar elementos como:

- **int** → palabra reservada.
- **edad** → identificador.
- **=** → operador.
- **20** → número.
- **;** → separador.

- 3. Análisis sintáctico

Se comprueba que las instrucciones están escritas siguiendo las reglas del lenguaje.

Por ejemplo:

```
int edad = 20;
```

es correcto.

Pero:

```
int = edad 20;
```

tiene un error de sintaxis.

Es parecido a comprobar si una frase está correctamente construida.

- 4. Análisis semántico

Se comprueba que el programa **tenga sentido**.

Por ejemplo:

```
int edad = "hola";
```

La sintaxis puede parecer correcta, pero existe un problema porque estamos intentando guardar un texto en una variable de tipo **int**.

- 5. Traducción o compilación

El compilador traduce el código fuente a otra representación que pueda ser utilizada para ejecutar el programa.

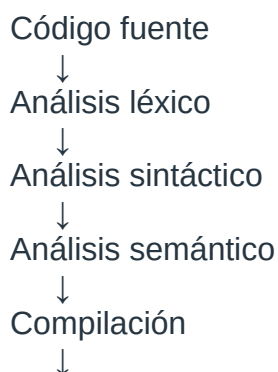
En Java:

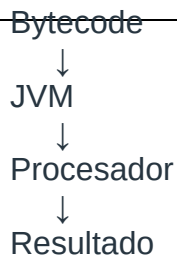
```
.java → compilador javac → .class
```

El archivo **.class** contiene **bytecode**.

- 6. Ejecución

Finalmente, la máquina virtual de Java (JVM) ejecuta el bytecode y el procesador realiza las instrucciones necesarias.





2. Clasificación de lenguajes de programación:

2.1 Investiga y clasifica los lenguajes de programación en función de: Nivel de abstracción: alto, medio y bajo.

Los lenguajes de programación pueden clasificarse según su **nivel de abstracción**, es decir, según lo cerca o lejos que estén del lenguaje que entiende directamente el ordenador.

Lenguajes de bajo nivel

Están muy cerca del lenguaje máquina y del funcionamiento interno del procesador.

Ejemplos:

Lenguaje máquina

Utiliza instrucciones representadas mediante números binarios.

Ensamblador (Assembly)

Utiliza instrucciones como:

```
MOV AX, 5  
ADD AX, 2
```

Estos lenguajes permiten controlar aspectos muy concretos del hardware, pero son más difíciles de programar.

Lenguajes de nivel medio

Intentan combinar características de bajo y alto nivel.

Ejemplo:

C

C permite trabajar con elementos cercanos al hardware, como memoria y direcciones, pero también permite utilizar estructuras más sencillas.

Otro ejemplo que suele clasificarse como de nivel medio es **C++**, aunque su clasificación puede variar según la fuente y el criterio utilizado.

Lenguajes de alto nivel

Están más alejados del funcionamiento interno del procesador y utilizan instrucciones más fáciles de entender para las personas.

Ejemplos:

Java

```
System.out.println("Hola");
```

Python

```
print("Hola")
```

También pertenecen a este grupo lenguajes como JavaScript, C# o PHP.

Resumen

Ejemplos

Características

Máquina,
Assembly

Muy cercanos al hardware

C, C++

Permiten trabajar con elementos de bajo nivel y estructuras de alto nivel

Java, Python

Más fáciles de leer y programar

Paradigma de programación: imperativo y declarativo.

2.2 Da al menos dos ejemplos de lenguajes para cada categoría y explica brevemente por qué pertenecen a esa clasificación.

Parte 2: Actividad práctica y de análisis

1. Identificación de paradigmas de programación a partir de ejemplos:

A continuación, se te ofrecen descripciones de fragmentos de código. Tu tarea es identificar si cada uno corresponde a un paradigma imperativo o declarativo y justificar tu respuesta.

Paradigma imperativo

En la programación imperativa, se indica al ordenador cómo debe realizar una tarea, utilizando una serie de instrucciones y pasos.

Es parecido a explicar una receta:

1. Coger los ingredientes.
2. Cortar los ingredientes.
3. Encender el fuego.

4. Cocinar.
5. Servir.

Ejemplos de lenguajes imperativos:

- Java.
- C.

Java es imperativo porque normalmente se indican las instrucciones que debe ejecutar el programa y el orden en el que debe hacerlo.

C también es imperativo porque el programador especifica las operaciones que tiene que realizar el programa.

Paradigma declarativo

En la programación **declarativa**, se indica principalmente **qué resultado queremos obtener**, sin explicar todos los pasos que debe realizar el ordenador para conseguirlo.

Ejemplos:

- SQL.
- Prolog.

Por ejemplo, en SQL podemos escribir:

```
SELECT nombre
```

```
FROM empleados
```

```
WHERE edad > 30;
```

Aquí indicamos que queremos obtener los nombres de los empleados mayores de 30 años, pero no tenemos que indicar cómo debe recorrer internamente la base de datos.

Fragmento 1 (Descripción):

o Un programa recorre una lista de números sumándolos uno por uno hasta obtener el total.

o Pista: Se describe cómo se realiza la suma paso a paso.

Un programa recorre una lista de números sumándolos uno por uno hasta obtener el total.

Clasificación: IMPERATIVO

Es **imperativo** porque se explica cómo realizar la operación.

Por ejemplo:

```
int total = 0;

for (int numero : numeros) {
    total = total + numero;
}
```

El programa:

1. Empieza con el total a 0.
2. Coge un número de la lista.
3. Lo suma al total.
4. Continúa con el siguiente número.
5. Repite el proceso hasta terminar.

Por tanto, se indican los pasos que debe realizar el programa.

Fragmento 2 (Descripción):

- Una consulta a una base de datos busca empleados mayores de 30 años y devuelve solo sus nombres.
- Pista: Se especifica qué resultado se quiere obtener sin detallar cómo se procesa internamente.

Una consulta a una base de datos busca empleados mayores de 30 años y devuelve solo sus nombres.

Clasificación: DECLARATIVO

Es **declarativo** porque indicamos qué información queremos obtener, pero no explicamos paso a paso cómo debe buscarla la base de datos.

Ejemplo:

```
SELECT nombre
FROM empleados
WHERE edad > 30;
```

Decimos:

~~"Quiero los nombres de los empleados cuya edad sea superior a 30 años."~~

El sistema de gestión de bases de datos decide cómo realizar internamente la búsqueda.

Fragmento 3 (Descripción):

- Un programa que calcula el factorial de un número n definiendo que el factorial de 0 es 1 y, para números mayores, multiplicando el número por el factorial del número anterior.
- Pista: La lógica se define recursivamente sin especificar los pasos detallados.

Un programa calcula el factorial de un número n definiendo que el factorial de 0 es 1 y, para números mayores, multiplicando el número por el factorial del número anterior.

Clasificación: DECLARATIVO

En el enunciado se define la relación matemática:

$$0! = 1$$

$$n! = n \times (n-1)!$$

La solución se describe mediante una **definición recursiva**, sin detallar necesariamente todos los pasos que debe ejecutar el ordenador.

Por eso, siguiendo la clasificación propuesta en la actividad, lo consideramos **declarativo**.

Fragmento 4 (Descripción):

- Un programa filtra productos con precios superiores a 10 dólares recorriendo una lista y comprobando cada producto uno por uno.
- Pista: Se describen detalladamente los pasos del proceso.

Un programa filtra productos con precios superiores a 10 dólares recorriendo una lista y comprobando cada producto uno por uno.

Clasificación: IMPERATIVO

Es **imperativo** porque se explican los pasos:

1. Recorrer la lista.
2. Coger un producto.
3. Comprobar su precio.
4. Comprobar si es superior a 10 dólares.
5. Guardarlo si cumple la condición.
6. Continuar con el siguiente producto.

Por tanto, se está indicando **cómo realizar la tarea**.

RESUMEN

Fragmento	Paradigma	Motivo
1	Imperativo	Indica los pasos para sumar
2	Declarativo	Indica que datos queremos obtener
3	Declarativo	Define el resultado mediante una relación recursiva
4	Imperativo	Indica los pasos a filtrar

2. Actividad en grupo:

o En equipos de 2-3 personas, seleccionen una actividad cotidiana (preparar una receta, organizar un evento, etc.) y describan la tarea de dos maneras: Imperativa: Indicando cada uno de los pasos detallados.

Para esta actividad hemos elegido una tarea cotidiana: **preparar una pizza**.

Descripción imperativa

En el enfoque imperativo indicamos todos los pasos que debemos realizar:

1. Preparar la masa.
2. Colocar harina en un recipiente.
3. Añadir agua, levadura y sal.
4. Mezclar los ingredientes.
5. Amasar hasta conseguir una masa uniforme.
6. Dejar reposar la masa.
7. Extender la masa.
8. Añadir tomate.
9. Añadir queso.
10. Añadir los ingredientes que queramos.

11. Precalentar el horno.
12. Introducir la pizza en el horno.
13. Esperar hasta que esté cocinada.
14. Sacar la pizza.
15. Servirla.

En este caso explicamos **cómo conseguir el resultado** paso a paso.

Declarativa: Describiendo únicamente el resultado final que quieren lograr.

Descripción declarativa

En el enfoque declarativo nos centramos en el resultado que queremos conseguir:

Obtener una pizza casera preparada y lista para comer, con masa, tomate, queso y los ingredientes elegidos.

No explicamos todos los pasos necesarios para prepararla.

o Comparen las dos descripciones y discutan las ventajas y desventajas de cada enfoque. Incluyan esta comparación en el informe.

Características	Imperativo	Declarativo
Se centra en	Cómo hacerlo	Qué queremos conseguir
Explica los pasos	Sí	No necesariamente
Control del proceso	Mayor	Menor
Facilidad para controlar cada operación	Alta	Menor
Código	Puede ser más largo	Puede ser más corto
Ejemplo	Java, C	SQL, Prolog

Ventajas del enfoque imperativo

- Permite controlar exactamente lo que hace el programa.
- Es útil cuando necesitamos indicar un orden concreto.

Permite modificar los datos paso a paso.

- Es adecuado para algoritmos que necesitan muchas operaciones.

Desventajas

- Puede necesitar más código.
- Los programas pueden ser más largos.
- El programador tiene que controlar muchos detalles.

Ventajas del enfoque declarativo

- Permite centrarse en el resultado.
- Puede necesitar menos código.
- Puede ser más sencillo expresar determinadas operaciones.
- El sistema puede encargarse de decidir cómo conseguir el resultado.

Desventajas

- Tenemos menos control sobre los pasos internos.
- No siempre es adecuado para cualquier problema.
- Puede ser más difícil controlar exactamente cómo se realiza una operación.

Conclusión

Durante esta actividad hemos aprendido que un programa informático pasa por diferentes fases desde que es escrito hasta que se ejecuta.

Primero tenemos el **código fuente**, que es el código escrito por el programador. Después se realiza un proceso de análisis y traducción para obtener una representación que pueda ser ejecutada. En el caso de Java, el código fuente se transforma en **bytecode**, que posteriormente es ejecutado por la **JVM**.

También hemos aprendido que los lenguajes pueden clasificarse según su **nivel de abstracción**, distinguiendo entre lenguajes de bajo, medio y alto nivel.

Además, hemos estudiado los paradigmas **imperativo y declarativo**. La principal diferencia es que la programación imperativa explica **cómo** realizar una tarea, mientras que la declarativa se centra principalmente en **qué resultado queremos obtener**.

Por último, hemos aplicado estos conceptos a ejemplos de programación y a una actividad cotidiana, comprobando que una misma tarea puede describirse de diferentes formas dependiendo del enfoque utilizado.

Entrega

- Una presentación formato de 5 a 10 páginas que contenga:
 - o Las respuestas a la Parte 1 (explicación de conceptos y clasificación de lenguajes).
 - o La clasificación y justificación de los ejemplos de la Parte 2.
 - o Una descripción del trabajo en grupo, incluyendo el ejemplo cotidiano y la comparación entre descripciones imperativas y declarativas.